Reinforcement Learning

Clase 12

Ignacio Brevis

ignaciobrve@unisabana.edu.co

Universidad de La Sabana

04 noviembre 2026



Recuerdo de la evaluación

Recordando la primera clase:

- Los estudiantes trabajarán durante el semestre en un **proyecto a elección**, aplicando las técnicas de RL aprendidas durante el semestre.
- Entrega de un informe **tipo paper** (en inglés), incluyendo: **(25 de noviembre)**
  - abstract
  - introducción
  - metodología
  - resultados
  - conclusiones
  - bibliografía
- Códigos reproducibles y comentados **(25 de noviembre)**.
- Presentación oral (en español) de ∼20 min. Describiendo el problema abordado, técnicas utilizadas, resultados obtenidos, etc. **(02 de diciembre)**
- **Se recomienda encarecidamente la participación durante el semestre**.

Importante: El informe debe estar escrito por el estudiante (no por un LLM).

Repaso: Deep Q-learning

- Primer trabajo en utilizar Deep NNs en RL de forma exitosa.
- Presentado en NIPS Deep Learning Workshop, 2013.
- En el 2015 fue publicado en Nature bajo el nombre “Human-level control through deep reinforcement learning” (vol. 518)
- Utilizando CNN logró jugar juegos de Atari (e incluso sobrepasar a humanos expertos)

Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih **Koray Kavukcuoglu** **David Silver** **Alex Graves** **Ioannis Antonoglou**
Daan Wierstra **Martin Riedmiller**
DeepMind Technologies
{vlad,koray,david,alex.graves,ioannis,daan,martin.riedmiller} @ deepmind.com

Abstract

We present the first deep learning model to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning. The model is a convolutional neural network, trained with a variant of Q-learning, whose input is raw pixels and whose output is a value function estimating future rewards. We apply our method to seven Atari 2600 games from the Arcade Learning Environment, with no adjustment of the architecture or learning algorithm. We find that it outperforms all previous approaches on six of the games and surpasses a human expert on three of them.



Figure 1: Screen shots from five Atari 2600 Games: (*Left-to-right*) Pong, Breakout, Space Invaders, Seaquest, Beam Rider

Image from Playing Atari with Deep Reinforcement Learning, 2013.

Repaso: Deep Q-learning

Deep Q-learning busca minimizar la función de costo:

$$J(w) = \frac{1}{2}\mathbb{E}\left[\left(\underbrace{R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; w)}_{\text{TD target}} - \hat{q}(S, A; w)\right)^2\right],$$

Este funcional busca minimizar el **Bellman optimality error**

$$\text{BOE}: \boxed{q(s,a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q(S_{t+1}, a') | S_t = s, A_t = a]}$$

Esto es, $R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; w) - \hat{q}(S, A; w)$ debe ser cero en bajo el valor esperado.

Repaso: Deep Q-learning

Dado el funcional:

$$\boxed{J(w,\eta) = \frac{1}{2}\mathbb{E}\left[\left(R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S',a;\eta) - \hat{q}(S,A;w)\right)^2\right]}$$

podemos calcular el gradiente fácilmente cuando η está fijo:

$$\nabla_w J(w,\eta) = \mathbb{E}\left[\left(R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S',a;\eta) - \hat{q}(S,A;w)\right)\nabla_w \hat{q}(S,A;w)\right]$$

- Experience replay
- Neural network structure

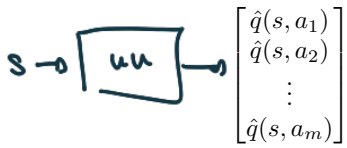
Repaso: Deep Q-learning

Deep Q-learning nos permite estimar la **acciones óptimas** por medio del algoritmo de TD:

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \max_{s' \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a'; \eta) - \hat{q}(s_t, a_t; w_t) \Big) \nabla_w \hat{q}(s_t, a_t; w_t)$$

donde $t = 0, 1, 2, \ldots$

- Actualiza η cada cierta cantidad de pasos. Entonces $\eta = w$
- Para cada t obtiene $\{(s_t, a_t, r_{t+1}, s_{t+1})\}$ desde un replay buffer (**experience replay**)
- La estructura de la red neuronal toma s como input, y devuelve



Repaso: Deep Q-learning

Algorithm 8.3: Deep Q-learning (off-policy version)

Initialization: A main network and a target network with the same initial parameter.
Goal: Learn an optimal target network to approximate the *optimal* action values from the experience samples generated by a given behavior policy π_b.

Store the experience samples generated by π_b in a replay buffer $\mathcal{B} = \{(s, a, r, s')\}$
  For each iteration, do
    Uniformly draw a mini-batch of samples from $\mathcal{B}$
    For each sample (s, a, r, s'), calculate the target value as $y_T = r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$, where w_T is the parameter of the target network
    Update the main network to minimize $(y_T - \hat{q}(s, a, w))^2$ using the mini-batch of samples
  Set $w_T = w$ every C iterations

(ver versión on-policy en página 181)

Page 184, Zhao, Mathematical foundations of reinforcement learning

Cautionary tale: Combinar estos tres elementos puede producir inestabilidades en los algoritmos:

- **Function approximation**: Aproximación lineal y no lineal.
- **Bootstrapping**: Como los algoritmos de temporal difference (TD) learning.
- **Off-policy training**: Entrenamiento con una política distinta de la producida por la política objetivo.

Ejemplo: El contraejemplo de Baird (ver Sección 11.2 de Sutton & Barto).

Obs: "Q-learning has never been found to diverge in this case, but there has been no theoretical analysis" (2018).

Sección 11.3, Sutton & Barto, 2018

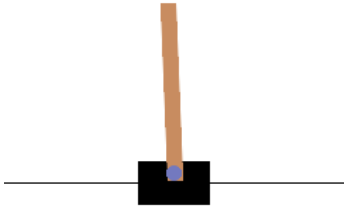
- Jianqing Fan, Zhaoran Wang, Yuchen Xie, Zhuoran Yang, **A Theoretical Analysis of Deep Q-Learning**, *Proceedings of Machine Learning Research*, Vol 120, 1–4, 2020.
- Zhikang T. Wang, Masahito Ueda, **Convergent and Efficient Deep Q Learning Algorithm**, Published as a conference paper at ICLR 2022.
- Shuai Zhang et al., **On the Convergence and Sample Complexity Analysis of Deep Q-Networks with ϵ-Greedy Exploration**, 37th Conference on Neural Information Processing Systems (NeurIPS 2023).

Deep Q-learning: Examples

See `https://gymnasium.farama.org/environments/classic_control/cart_pole/` for a guided introduction to the environment

In `https://github.com/ibrevis/RL_unisabana_2026_2`

- go to `/Lesson11/`
- Go `01_DQN_CartPole.ipynb` and tune the hyperparameters to traing the policy.
- Check your results with `run_dqn_cartpole_policy.py`.

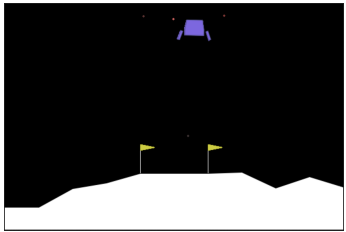


Deep Q-learning: Examples

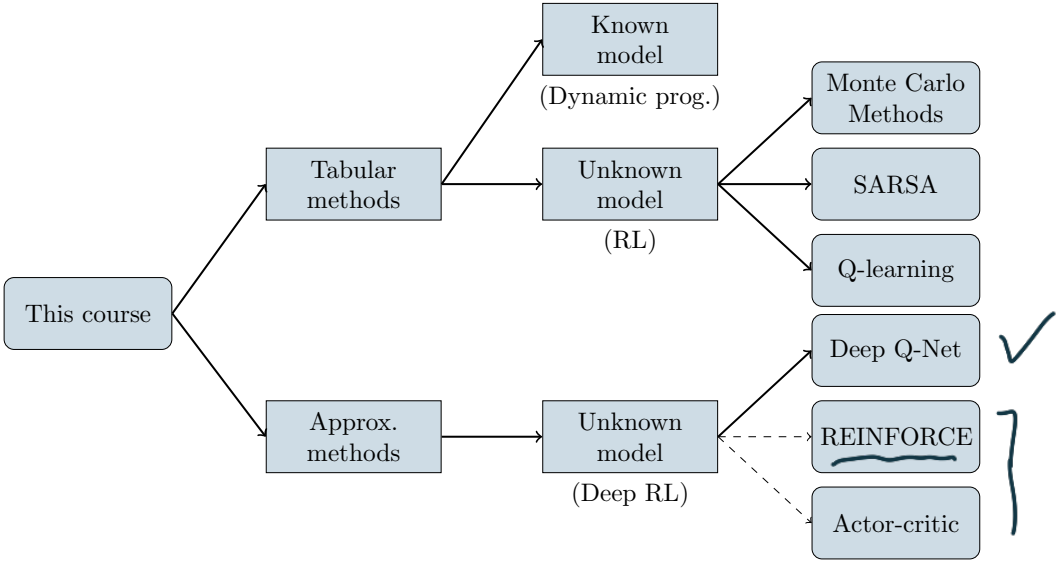
See `https://gymnasium.farama.org/environments/box2d/lunar_lander/` for a guided introduction to the environment

In `https://github.com/ibrevis/RL_unisabana_2026_2`

- go to `/Lesson11/`
- Go `02_DQN_LunarLander.ipynb` and complete the function `train_step()` to traing the policy.
- Check your results with `run_dqn_lunarlander_policy.py`.



Contenido


This course
Tabular methods
Approx. methods
Known model
(Dynamic prog.)
Unknown model
(RL)
Unknown model
(Deep RL)
Monte Carlo Methods
SARSA
Q-learning
Deep Q-Net
REINFORCE
Actor-critic

Policy gradient methods

En los casos anteriores, representamos las políticas en forma de tablas, donde podíamos cambiar la política de un estado independientemente de los otros estados

	a_1	a_2		a_m
s_1	$\pi(a_1\|s_1)$	$\pi(a_2\|s_1)$	$\dots$	$\pi(a_m\|s_1)$
s_2	$\pi(a_1\|s_2)$	$\pi(a_2\|s_2)$	$\dots$	$\pi(a_m\|s_2)$
$\vdots$	$\vdots$	$\vdots$		$\vdots$
s_n	$\pi(a_1\|s_n)$	$\pi(a_2\|s_n)$	$\dots$	$\pi(a_m\|s_n)$

$\sum_{i=1}^{m} \pi(a_i|s_k) = 1$

value-based methods

↓

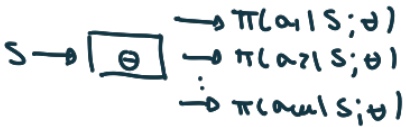
policy-based methods

$\pi(a|s;\theta)$

$\theta \in \mathbb{R}^d$ los parám. a entrenar

Ahora representamos la política con una función parametrizada por $\theta \in \mathbb{R}^d$

$$\pi(a|s;\theta)$$



- La función puede ser una red neuronal con input s y salida la probabilidad de cada acción. ✓
- Cuando el espacio de estados $\mathcal{S}$ es muy grande, la representación tabular no es eficiente!
- Al igual que en caso anterior, ya no podemos modificar la política para un estado en particular. Al actualizar θ, modificamos la política de todos (o varios) estados.

Policy gradient methods

- Hasta ahora, todos los métodos se han basado en la **action-value function** (las políticas se calculan en base al valor de las acciones $q(s, a)$).
- Ahora consideraremos métodos donde la **política parametrizada** aprende sin necesidad de consultar directamente una función de valor (al menos para la selección de la acción).

$V_{\pi'}(s) \geq V_{\pi}(s) \quad \forall s \in S$

$\pi(a|s;\theta) \leftarrow J(\theta)$

notación

$\pi(a, s; \theta)$

$\pi_\theta(a|s)$

$\pi_\theta(a, s)$

Policy gradient methods

Idea detrás de los métodos de **policy gradient**:

- Primero debemos definir la función objetivo (o métrica) $J(\theta)$ para obtener políticas óptimas.
- Segundo, debemos calcular su gradiente $\nabla_\theta J(\theta)$ para utilizar un algoritmo de optimización basado en métodos de "gradient descent"

Aunque la idea parece simple, la respuesta no es fácil de obtener.

- Cuál es la métrica apropiada?
- Cómo calculamos el gradiente asociado a la métrica?

Policy gradient methods: Metrics

Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a "surrogate" objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

2 Background: Policy Optimization

2.1 Policy Gradient Methods

Policy gradient methods work by computing an estimator of the policy gradient and plugging it into a stochastic gradient ascent algorithm. The most commonly used gradient estimator has the form

$$\hat{g} = \hat{\mathbb{E}}_t\Big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\hat{A}_t\Big] \tag{1}$$

where π_θ is a stochastic policy and $\hat{A}_t$ is an estimator of the advantage function at timestep t. Here, the expectation $\hat{\mathbb{E}}_t[\ldots]$ indicates the empirical average over a finite batch of samples, in an algorithm that alternates between sampling and optimization. Implementations that use automatic differentiation software work by constructing an objective function whose gradient is the policy gradient estimator; the estimator $\hat{g}$ is obtained by differentiating the objective

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t\Big[\log \pi_\theta(a_t \mid s_t)\hat{A}_t\Big]. \tag{2}$$

While it is appealing to perform multiple steps of optimization on this loss L^{PG} using the same trajectory, doing so is not well-justified, and empirically it often leads to destructively large policy updates (see Section 6.1; results are not shown but were similar or worse than the "no clipping or penalty" setting).

Metric: Average state value

$$\bar{v}_\pi := \mathbb{E}_{S\sim d}\left[v_\pi(S)\right] = \sum_s d(s) v_\pi(s) = d^\top v_\pi$$

donde $d(s)$ es el peso/probabilidad del estado s.

- $d(s) \geq 0$ para cada $s \in \mathcal{S}$.
- $\sum_{s\in\mathcal{S}} d(s) = 1$

$$d = \begin{bmatrix} d(s_1) \\ \vdots \\ d(s_n) \end{bmatrix}$$

$$v_\pi = \begin{bmatrix} v_\pi(s_1) \\ \vdots \\ v_\pi(s_n) \end{bmatrix}$$

Metric: Average state value

¿Cómo elegir d en $\bar{v}_\pi$?

d uniforme

$d(s) = \frac{1}{|S|} = \frac{1}{u}$ $\quad |S| = u$

nos interesa un estado en part.

$d(s_0) = 1$

$d(s) = 0 \quad \forall s \neq s_0$

En este caso

$\bar{v}_\pi = v_\pi(s_0)$

d es independiente de la política

d.st. estac.

$d = d_\pi$ $\qquad d_\pi^T P_\pi = d_\pi^T$

(matriz trans.)

depende de la política.

Metric: Average state value

$$J(\theta) := \lim_{n\to\infty} \mathbb{E}\left[\sum_{t=0}^{n} \gamma^t R_{t+1}\right] = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right] = \sum_{s\in S} d(s)\, \mathbb{E}\left(\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s\right)$$

$$= \sum_{s\in S} d(s)\, \mathbb{E}(G_t \mid S_0 = s) = \sum_{s\in S} d(s) v_\pi(s)$$

$$= \bar{v}_\pi$$

Policy gradient methods: Metrics

Metric: Average (one-step) reward

$$\bar{r}_\pi = r(\pi) := \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1} | S_0 = s_0\right] = \lim_{n\to\infty} \frac{1}{n}\sum_{t=0}^{n-1} \mathbb{E}(R_{t+1} | S_0 = s_0)$$

$$= \lim_{t\to\infty} \mathbb{E}(R_{t+1} | S_0 = s_0)$$

Promedio de Cesaro

$(a_n)_{n=1}^{\infty}$ conv.

$a_n \xrightarrow[n\to\infty]{} L$

$\Rightarrow \left(\frac{1}{n}\sum_{k=1}^{n} a_k\right)$ conv.

$\frac{1}{n}\sum_{k=1}^{n} a_k \to L$

Metric: Average (one-step) reward (continuación)

$$\bar{r}_\pi = r(\pi) := \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1} | S_0 = s_0\right] = \lim_{t\to\infty} \mathbb{E}(R_{t+1} | S_0 = s_0)$$

$$= \lim_{t\to\infty} \sum_s \mathbb{E}(R_{t+1} | S_0 = s_0, S_t = s)\, p^{(t)}(s|s_0) \quad \rightarrow \text{prob. en } t \text{ pasos}$$

$$= \lim_{t\to\infty} \sum_s \underbrace{\mathbb{E}(R_{t+1} | S_t = s)}_{r_\pi(s)}\, p^{(t)}(s|s_0)$$

$$= \lim_{t\to\infty} \sum_s r_\pi(s)\, p^{(t)}(s|s_0) = \boxed{\sum_s r_\pi(s)\, d_\pi(s)} \quad \text{dist. estac.}$$

donde $r_\pi(s) = \sum_a \pi(a|s)\, r(s,a)$

Metrics summary

Métrica	suma	valor esperado	límite
$\bar{v}_\pi$	$\sum_{s\in\mathcal{S}} d(s)v_\pi(s)$	$\mathbb{E}_{S\sim d}\left[v_\pi(S)\right]$	$\lim_{n\to\infty} \mathbb{E}\left[\sum_{t=0}^{n} \gamma^t R_{t+1}\right]$
$\bar{r}_\pi$	$\sum_{s\in\mathcal{S}} d_\pi(s)r_\pi(s)$	$\mathbb{E}_{S\sim d_\pi}\left[r_\pi(S)\right]$	$\lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1}\right]$

donde $r_\pi(s) = \sum_{a\in\mathcal{A}(s)} \pi(a|s)r(s,a) = \mathbb{E}_{A\sim\pi(s)}\left[r(S,A)|S=s\right]$ $\left(r(s,a) = \sum_r rp(r|s,a)\right)$

Policy gradient methods: Gradients

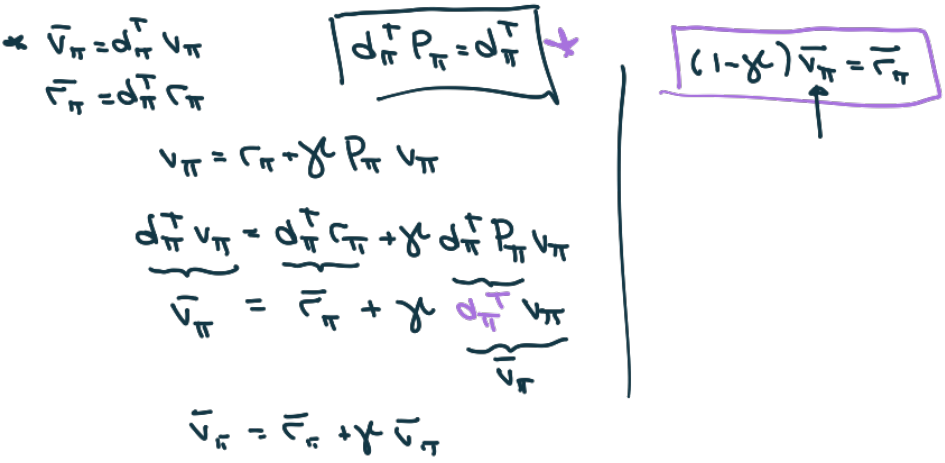
Métrica	suma	valor esperado	límite
$\bar{v}_\pi$	$\sum_{s\in\mathcal{S}} d(s)v_\pi(s)$	$\mathbb{E}_{S\sim d}\left[v_\pi(S)\right]$	$\lim_{n\to\infty} \mathbb{E}\left[\sum_{t=0}^{n} \gamma^t R_{t+1}\right]$
$\bar{r}_\pi$	$\sum_{s\in\mathcal{S}} d_\pi(s)r_\pi(s)$	$\mathbb{E}_{S\sim d_\pi}\left[r_\pi(S)\right]$	$\lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1}\right]$

- Estas métricas son funciones de π (por lo tanto, de la parametrización θ).
- Las funciones de pérdida $J(\theta)$ estarán definidas en términos de estas métricas. Entonces, nuestro problema se traduce en **buscar θ que maximice estas métricas**.
- Calcular los gradientes es una de las partes más complejas de los métodos de policy gradient, por lo que debemos distinguir varios casos!

Policy gradient methods: Gradients (caso con descuento)

Caso con descuento ($\gamma < 1$)

Notemos que ambas métricas son equivalentes para el caso de la distribución estacionaria:



Policy gradient methods: Gradients (caso con descuento)

Caso con descuento ($\gamma < 1$)

Después de algunas cuentas...

$$\nabla_\theta v_\pi(s) = \sum_{s' \in \mathcal{S}} \Big[(I_n - \gamma P_\pi)^{-1} \Big]_{ss'} \sum_{a \in \mathcal{A}} \nabla_\theta \pi(a|s';\theta) q_\pi(s', a),$$

donde $[(I_n - \gamma P_\pi)^{-1}]_{ss'} = \sum_{k=0}^{\infty} \gamma^k [P_\pi^k]_{ss'}$.

- $[P_\pi]_{ss'}$ es la probabilidad de transición de s a s' (en un paso).
- $[P_\pi^k]_{ss'}$ es la probabilidad de transición de con k pasos.
- $\Pr_\pi(s'|s) := [(I_n - \gamma P_\pi)^{-1}]_{ss'}$ es la probabilidad de transición con cualquier número de pasos.

Detalles en Zhao, Mathematical foundations of RL, Box 9.2: Proof of Lemma 9.2

Policy gradient methods: Gradients (caso con descuento)

$$\bar{v}_\pi := \mathbb{E}_{S\sim d}\Big[v_\pi(S)\Big] = \sum_{s\in\mathcal{S}} d(s) v_\pi(s)$$

(average state value)

Caso cuando $d = d_0$ (independiente de la política)

Theorem (Gradiente de $\bar{v}_\pi^0$)

En el caso donde $\gamma \in (0,1)$ *y* $d = d_0$ *es independiente de la política, el gradiente de* $\bar{v}_\pi := \bar{v}_\pi^0$ *viene dado por:*

$$\nabla_\theta \bar{v}_\pi^0 = \mathbb{E}\Big[\nabla_\theta \ln \pi(A|S;\theta) q_\pi(S,A)\Big],$$

donde (para el gradiente) $S \sim \rho_\pi$ *y* $A \sim \pi(S;\theta)$. *Aquí* ρ_π *denota:*

$$\rho_\pi(s) = \sum_{s'\in\mathcal{S}} d_0(s') \Pr_\pi(s|s'), \quad s \in \mathcal{S}.$$

Policy gradient methods: Gradients (caso con descuento)

$$\bar{v}_\pi := \mathbb{E}_{S\sim d}\Big[v_\pi(S)\Big] = \sum_{s\in\mathcal{S}} d(s) v_\pi(s)$$

(average state value)

$$\nabla \bar{v}_\pi = \sum_s \nabla d_\pi(s)\, v_\pi(s) + \sum_s d_\pi(s) \nabla v_\pi(s)$$

$\nabla d_\pi(s) = 0$

$\nabla v_\pi(s)$ conocido!

Caso cuando $d = d_\pi$ (distribución estacionaria)

Theorem (Gradiente de $\bar{v}_\pi$)

En el caso donde $\gamma \in (0,1)$ y $d = d_\pi$, el gradiente de $\bar{v}_\pi$ viene dado por:

$$\nabla_\theta \bar{v}_\pi \approx \frac{1}{1-\gamma} \sum_{s\in\mathcal{S}} d_\pi(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a|s;\theta) q_\pi(s,a)$$

$$= \frac{1}{1-\gamma} \mathbb{E}\Big[\nabla_\theta \ln \pi(A|S;\theta) q_\pi(S,A)\Big],$$

donde $S \sim d_\pi$ y $A \sim \pi(S;\theta)$.

cuando γ es aprox. 1.

Policy gradient methods: Gradients (caso sin descuento)

Caso sin descuento ($\gamma = 1$)

- Redefinimos los valores de estado y de acciones:

$$v_\pi(s) = \mathbb{E}\Big[(R_{t+1} - \bar{r}_\pi) + (R_{t+2} - \bar{r}_\pi) + \ldots | S_t = s\Big],$$

$$q_\pi(s) = \mathbb{E}\Big[(R_{t+1} - \bar{r}_\pi) + (R_{t+2} - \bar{r}_\pi) + \ldots | S_t = s, A_t = a\Big].$$

- La ecuación de Bellman es "reemplazada" por la ecuación de Poisson

$$v_\pi = r_\pi - \bar{r}_\pi \mathbf{1}_n + P_\pi v_\pi.$$

- El gradiente viene dado por:

$$\nabla_\theta \bar{r}_\pi = \mathbb{E}\Big[\nabla_\theta \ln \pi(A|S;\theta) q_\pi(S,A)\Big],$$

donde $S \sim d_\pi$ y $A \sim \pi(S;\theta)$.

Detalles en Zhao, Mathematical foundations of RL, Section 9.3.2

Policy gradient methods: Gradients

Resumen:

	Discounted	Undiscounted
$\bar{v}_\pi$ defined?	Yes, unique	Non-unique
$\bar{r}_\pi$ defined?	Yes	Yes
$\bar{r}_\pi = (1-\gamma)\bar{v}_\pi$	Holds ✔	Breaks down ✗
Policy gradient of $\bar{r}_\pi$	Approximate	Exact
State values satisfy	Bellman equation	Poisson equation

Para maximizar $J(\theta)$ utilizaremos gradient ascent. En todos los casos, podemos asumir que gradiente viene dado por:

$$\nabla_\theta J(\theta) \quad \text{“}=\text{”} \quad \mathbb{E}\Big[\nabla_\theta \ln \pi(A|S;\theta) q_\pi(S,A)\Big]$$